

The Structure of 'C' Program

Understanding the six components that make up a well-structured C program



Documentation

Program description and metadata



Preprocessor

Header files and definitions



Global Declaration

Variables and functions

Introduction to C Program Structure

A 'C' program's basic structure is separated into six components, making it easier to read, edit, document, and comprehend in a specific manner.

Benefits of Proper Structure



Readability

Well-structured code is easier to read and understand



Editability

Organized code is simpler to modify and maintain



Debugging

In a well-structured C program, debugging is easy

```
// Example of a basic C program structure
/*
 * Documentation Section
 */
#include <stdio.h> // Preprocessor Section
#define PI 3.14159 // Definition Section

int add(int a, int b); // Global Declaration

int main() { // Main Function
    // Function body
    return 0;
}
```

```
int add(int a, int b) { // Sub Program
    return a + b;
}
```

2/8

Documentation Section

This part includes a program description, the program's name, and the program's inception date and time.

Purpose of Documentation

It is supplied in the form of comments at the beginning of the program. Essentially, it provides the reader with an overview of the software.

```
// Single-line comment style
// description, name of the program, programmer name, date, time etc.

// Multi-line comment style
/*
 * description, name of the program, programmer name, date, time etc.
 */

// Example of proper documentation
/**
 * file: calculator.c
 * author: John Doe
 * description: A simple calculator program
 * date: 2023-06-15
 */
```



Key Point

Anything placed in comments will be viewed as program documentation and will not interfere with the supplied code.

Preprocessor and Definition Sections

These sections handle header files and constant definitions before compilation.

Preprocessor Section

All of the program's header files will be defined in the program's preprocessing section. Header files allow us to incorporate better code from others into our own.

```
#include<stdio.h>
#include<math.h>
#include<stdlib.h>
```

Definition Section

The `#define` preprocessor is used to declare a constant that will be used throughout the program. When the compiler encounters this term, it replaces it with the actual piece of declared code.

```
#define long long ll
#define PI 3.14159
#define MAX_SIZE 100
```

How It Works

Preprocessors are programs that process our source code prior to compilation. A copy of these various files is placed into our software

before the compilation process. Preprocessor instructions begin with the symbol '#'.

4/8

Global Declaration and Main Function



These sections define global variables and the entry point of the program.

Global Declaration

Global variables, function declarations, and static variables are all included in the global declaration section. Variables and functions specified in this scope can be utilized throughout the application.

```
int num = 18;
float average;
int sum(int x, int y);
```

Main() Function

A main function must be present in every C program. This section contains the program's main() function. Declaration and implementation are carried out within the curly braces in the main program.

```
void main()
int main(void)
int main(int argc, char *argv[])
```

Return Types

The return type of the main() function can be either int or void. The compiler is informed by void() main that the program will not produce

any value. The `main()` function instructs the compiler that the code will return a numerical value.

5/8

Sub Programs Section

In this area of the program, user-defined functions are invoked and defined.

Function Control Flow

When user-defined functions are called from the `main()` function or from outside the `main()` function, control of the program is transferred to the called function. These are specified in accordance with the programmer's needs.

```
// Example of sub programs
#include <stdio.h>

// Function declaration (in global section)
int sum(int x, int y);
int multiply(int x, int y);

int main() {
    int a = 5, b = 10;
    printf("Sum: %d\n", sum(a, b));
    printf("Product: %d\n", multiply(a, b));
    return 0;
}

// Function definitions (sub programs)
int sum(int x, int y) {
    return x + y;
}

int multiply(int x, int y) {
    return x * y;
}
```



Benefits of Sub Programs

Sub programs help in code reusability, modularity, and better organization. They break down complex problems into smaller, manageable pieces, making the code easier to understand and maintain.

6/8

Complete Example and Compilation

Let's examine a complete C program that finds the sum of 2 numbers and understand the compilation process.

```
// Documentation
/**
 * file: sum.c
 * description: program to find sum.
 */

// Link
#include <stdio.h>

// Definition
#define X 20

// Global Declaration
int sum(int y);

// Main() Function
int main(void) {
    int y = 55;
    printf("Sum: %d", sum(y));
    return 0;
}

// Subprogram
int sum(int y) {
    return y + X;
}
```

Compilation and Execution Process

- 1 **Program Creation** - Writing the source code with proper structure
- 2 **Compilation** - Converting source code to machine-readable code
- 3 **Execution** - Running the compiled program
- 4 **Output** - Displaying the results of the program

Output

When the above program is compiled and executed, it will output:

Sum: 75

7/8

Syntax of C Program

The C standard specifies a formal grammar for C with specific rules for statements, expressions, and control flow.

Comments in C

Comments can appear within the delimiters `/*` and `*/`, or (since C99) after `//` until the end of the line. Comments delimited by `/*` and `*/` do not nest, and if they occur inside string or character literals, they are not interpreted as comment delimiters.

Declarations and Statements

The declarations and defined functions are found in C source files. Declarations and statements are included in function definitions. Declarations either declare new types with keywords like `struct`, `union`, and `enum`, or assign types to and maybe reserve storage for new variables by writing a type followed by the variable name.

```
// Example of C syntax elements
#include <stdio.h>

// Type declaration typedef struct {
    int x;
    int y;
} Point;

// Function declaration int calculate_distance(Point p1, Point p2);

// Control flow statements if (condition) {
    // statements
} else {
    // statements
}

for (int i = 0; i < 10; i++) {
    // loop statements
}

switch (expression) {
    case 1:
        // statements
        break;
    default:
        // statements
}
```

Key Takeaways

- A C program's basic structure is separated into six components 
- In a well-structured C program, debugging is easy 
- C provides various control-flow statements for structured programming 
- Expressions in C can employ a variety of integrated operators 
- Sequence points determine when side effects occur 